

高阶四边形单元有限元程序 - README 运行说明

一、项目简介

本程序是高阶四边形等参单元有限元程序设计作业的完整实现，基于 Python 与 NumPy 开发，无需 SciPy 等额外依赖。程序实现了 Q4 双线性单元、Q8 Serendipity 单元、Q9 双二次 Lagrange 单元的形函数构造、等参映射、高斯积分与有限元求解全流程，并通过 6 组算例验证单元的数值特性。

二、运行环境与依赖

2.1 软件环境

- Python 3.8 及以上版本
- 操作系统：Windows / macOS / Linux 均可

2.2 Python 依赖库

库名	版本要求	用途
numpy	≥ 1.20	数值计算、矩阵运算
matplotlib	≥ 3.0	结果可视化、绘图

注意：程序不依赖 SciPy，仅使用 NumPy 和 Matplotlib 即可运行。

2.3 安装依赖

如果尚未安装依赖，可通过 pip 安装：

```
bash
pip install numpy matplotlib
```

三、文件与模块说明

3.1 程序文件

- 高阶单元程序设计作业.py — 主程序文件，包含全部代码

3.2 代码模块结构

模块	主要函数	功能说明
1. 高斯积分工具	gauss_legendre_1d, gauss_quad_2d, get_standard_quad_order	一维/二维高斯积分点与权重生成，支持 1~4 阶
2. 单元形函数核心	get_natural_nodes, shape_quad	Q4/Q8/Q9 形函数值与自然坐标导数计算
3. 等参映射与 Jacobian	compute_jacobian, shape_deriv_physical	Jacobian 矩阵计算、物理坐标导数转换
4. 网格生成	generate_linear_grid, upgrade_to_high_order	结构化网格生成、Q4 网格升级为 Q8/Q9
5. 系统组装与求解	assemble_stiffness_and_load, apply_dirichlet_bc, solve_poisson_problem	总刚组装、边界条件施加、Poisson 方程求解
6. 静力凝聚	static_condensation_element	Q9 单元内部自由度凝聚消去
7. 验证算例	test1 ~ test6	6 组验证算例，依次执行并输出结果

四、运行操作步骤

4.1 直接运行

在终端/命令行中进入程序所在目录，执行：

```
bash
```

```
python 高阶单元程序设计作业.py
```

程序会自动依次执行全部 6 个算例，并在控制台输出计算结果，同时生成 3 张结果图片。

4.2 使用 IDE 运行

如果使用 PyCharm、VS Code 等 IDE：

1. 打开程序文件
2. 确认 Python 解释器已配置好 numpy 和 matplotlib
3. 点击运行按钮（或按 Shift+F10）

4.3 运行耗时

全部算例运行时间约 5~10 秒（取决于计算机性能），无需等待很久。

五、程序输出说明

5.1 控制台输出

程序运行后，会在控制台依次输出 6 个算例的计算结果：

1. **算例 1：**形函数 Kronecker delta 性质检验结果（通过/失败）
2. **算例 2：**单位分解与导数和为零的最大误差（机器精度量级）
3. **算例 3：**各类单元最小 Jacobian 行列式（均大于 0 为合法）
4. **算例 4：**线性场与二次场重构最大误差（验证完备性）
5. **算例 5：**Poisson 方程收敛性数据表 + 收敛阶 + 完全/减缩积分对比
6. **算例 6：**Q9 单元静力凝聚前后规模对比 + 边界解差异验证

5.2 生成的图片文件

程序运行完成后，会在当前目录下生成 3 张 PNG 格式图片，可直接用于作业报告：

文件名	内容说明	对应章节
poisson_solution_error_compariso	8×8 网格下 Q4/Q8/Q9 三种单元的	第 6.2

n.png	数值解云图与绝对误差分布（2 行 3 列）	节
convergence_curve.png	Q4/Q8/Q9 单元收敛性对比曲线（双对数坐标）	第 6.3 节
condition_number.png	Q4/Q8/Q9 单元刚度矩阵条件数对比（半对数坐标）	第 6.4 节

图片分辨率为 150 DPI，可直接插入 Word 文档使用。

六、算例介绍与预期结果

6.1 算例 1：Kronecker delta 性质检验

检验内容：在每个单元节点处计算形函数值，验证“本节点形函数为 1，其余节点为 0”。

预期结果：Q4、Q8、Q9 全部通过。

6.2 算例 2：单位分解与导数和为零检验

检验内容：随机选取 10 个采样点，验证 $\sum N_i=1$ 、 $\sum \partial N_i / \partial \xi = 0$ 、 $\sum \partial N_i / \partial \eta = 0$ 。

预期结果：所有误差均在 $1e-15$ 量级（机器精度）。

6.3 算例 3：Jacobian 行列式检验

检验内容：规则矩形、非规则四边形、曲边单元三类几何，验证所有积分点 $\det(J) > 0$ 。

预期结果：所有单元最小行列式均大于 0，映射合法。

6.4 算例 4：完备性检验（Patch Test）

检验内容：直边单元下，验证线性场 $u=1+2x-3y$ 和二次场 $u=x^2+xy+y^2$ 的重构精度。

预期结果：

- 线性场：三类单元误差均为机器精度（线性完备）
- 二次场：Q4 误差约 0.5（仅双线性），Q8/Q9 误差为机器精度（二次完备）

6.5 算例 5：Poisson 方程收敛性分析

算例设置：单位正方形域，精确解 $u=\sin(\pi x)\sin(\pi y)$ ， 4×4 / 8×8 / 16×16 三套网格。

预期结果：

- Q4 收敛阶 ≈ 2 (二阶收敛)
- Q8 收敛阶 ≈ 4 (四阶收敛)
- Q9 收敛阶 ≈ 4 (四阶收敛, 精度略优于 Q8)

6.6 算例 6：Q9 单元静力凝聚验证

检验内容：凝聚消去 Q9 内部节点，对比凝聚前后边界节点解的差异。

预期结果：边界节点解最大差异为 $1e-16$ 量级 (机器精度)，凝聚完全精确。

七、关键程序规则与注意事项

7.1 单元节点编号顺序

所有单元节点均按逆时针顺序编号，角节点在前，边中节点次之，内部节点最后。请勿随意调整节点顺序，否则会导致形函数计算错误或 Jacobian 行列式为负。

7.2 积分阶数选择

单元类型	完全积分	减缩积分
Q4	2×2	1×1
Q8	3×3	2×2
Q9	3×3	2×2

减缩积分可减少计算量，但存在零能模式（沙漏模式）风险，需配合边界约束或沙漏抑制使用。

7.3 边界条件施加方式

采用置 1 法施加 Dirichlet 边界条件：先修正右端项，再将边界行/列置零，对角线置 1。该方法保持矩阵对称性，数值稳定性好。

7.4 中文显示兼容

程序已内置多套中文字体 fallback (SimHei、Microsoft YaHei、DejaVu Sans)，并关闭了 Unicode 减号的数学文本模式，兼容各版本 matplotlib，不会出现中文乱码或字

体警告。

八、常见问题与解决方法

8.1 运行报错：ModuleNotFoundError

现象：提示 "No module named numpy" 或 "No module named matplotlib"

解决：安装缺失的依赖库

```
bash
pip install numpy matplotlib
```

8.2 图片中中文显示为方框

现象：生成的图片中文字变成方块或乱码

原因：系统中缺少 SimHei 或 Microsoft YaHei 字体

解决：程序已自动 fallback 到 DejaVu Sans，若仍有问题可安装中文字体，或在代码中修改 font.sans-serif 配置。

8.3 运行很慢或卡住

现象：程序运行时间远超预期

可能原因：网格过密或使用了过高阶积分

解决：本程序默认网格最大为 16×16，运行时间应在 10 秒内。若自行修改了网格参数，请注意控制规模。

8.4 报错：非法单元 $\det(J) \leq 0$

现象：抛出 ValueError: 非法单元: $\det(J) \leq 0$

原因：单元几何严重畸变，节点顺序错误，或单元发生了自交

解决：检查节点编号顺序是否为逆时针，网格畸变程度是否过大。

8.5 图片保存失败

现象：提示权限不足或路径错误

解决：确保程序在有写入权限的目录下运行，或修改 savefig 的保存路径。

九、自定义与扩展

9.1 修改算例参数

可在 `test5_poisson_convergence` 函数中修改：

- `grids = [4, 8, 16]` — 修改网格密度列表
- `u_exact / f_func` — 修改精确解与右端项

9.2 调用求解器接口

程序提供了 `solve_poisson_problem` 函数，可直接调用求解自定义问题：

```
python
result = solve_poisson_problem(
    nx=8, ny=8,          # 网格密度
    element_type="Q9",    # 单元类型：Q4 / Q8 / Q9
    u_exact=u_exact_func, # 精确解函数
    f_func=f_func,        # 右端项函数
    reduced=False         # 是否使用减缩积分
)
print(result["l2_error"])
print(result["u"]) # 节点解
```

9.3 返回结果字段说明

字段	类型	说明
coords	ndarray (N,2)	节点坐标数组
connect	ndarray (M,nen)	单元连接关系矩阵
u	ndarray (N,)	数值解
u_exact	ndarray (N,)	节点精确解
max_error	float	最大节点误差
l2_error	float	L2 相对误差

cond	float	刚度矩阵条件数
nnz	int	非零元数量
nnodes	int	总节点数
nelems	int	总单元数
K	ndarray (N,N)	施加边界条件后的总刚矩阵
